# SOURCE AUDIT & VERSION COMPARISON

De Rueda Construction ERP • Previous ZIP vs. Newer ZIP • 22 October 2026

**Executive conclusion: The newer archive improves the front-end credential helper, removes much of the seeded employee/payroll sample data, adds the PayMongo setup dialog, refreshes the messaging UI, and adds standard static files. However, it also introduces unauthenticated backup and PayMongo configuration endpoints and an automatic local-data deletion routine. Those are release-blocking risks. Do not deploy Version B for real payroll or payments until the P0 issues are fixed.**

## 1. What was compared

| Item | Version A — previous | Version B — newly uploaded |
|---|---|---|
| Archive name | DE-RUEDA-main (8).zip | DE-RUEDA-main (1).zip |
| Archive size | 24.1 MB | 27.1 MB |
| Files inside project root | 111 | 118 |
| Raw ZIP entries | 144 | 153 |
| Comparison result | Baseline source | 7 added files; 22 existing files changed; no project files removed |
| package.json / package-lock.json | Present; root name/version align | Unchanged; root name/version align; no Bun lockfile found |

### File hashes (to identify the exact archives)

**Version A SHA-256:** `b891a575f74d2049050affddd51b7c6e6db8ddc6058a26fd4e980fce244bbe63`

**Version B SHA-256:** `c50d32d44fef27783d01a96150f09c8eaaefdab60375e3000949c1e82aae32bd`

**This is a source-archive comparison. The fresh browser audit could not load emybuilder.pages.dev because of ERR_QUIC_PROTOCOL_ERROR; the same error affected unrelated control sites. Thus, these findings describe the uploaded source and are not proof of the exact code currently running live.**

## 2. At-a-glance comparison

| Area | What Version B improves | Remarks / regression / unresolved risk |
|---|---|---|
| **Front-end credentials** | Removes plaintext passwords and MFA/backup values from `src/utils/credentials.ts`; sanitizes account records before local storage. | Partial fix only. `src/server/auth.ts` still contains fixed demo accounts, password hashes derived from predefined passwords, known TOTP seeds and backup-code hashes; `server.ts` still has a request-header role fallback. |
| **Initial/sample data** | Sets core starter arrays (employees, sites, attendance, payroll, messages, and payment records) to empty. | Useful for a clean first install, but Version B automatically deletes both V1 and V2 browser data keys on launch. Also some sample equipment and login-directory/audit entries remain. |
| **PayMongo configuration UI** | Adds a dedicated four-field environment dialog with test/live selector and masked credential fields. | New endpoints lack authentication, do not persist secrets durably, and the “validate” action checks key prefixes instead of contacting PayMongo. Webhook-secret variable name is inconsistent. |
| **Chatbox** | Adds mobile list→chat navigation, details drawer, textarea composer, file-size check and an attachment preview/progress state. | Attachment progress is simulated with a timer; actual file bytes are not uploaded. Messages still use client state/localStorage, not shared durable messaging. UI says “Verified Enterprise Channel” without evidence of actual verification. |
| **Navigation / UI** | Reorganizes sidebar labels/sections and gives customer payments a clearer named area. | Still many overlapping operational destinations; change does not remove the underlying redundant payment systems. Need responsive and accessibility testing. |
| **Public static files** | Adds robots.txt, sitemap.xml, security.txt and manifest.json. | Files reference deruedaconstruction.com although the public audit target was emybuilder.pages.dev. Manifest points to /favicon.ico but no favicon file exists in the ZIP. |
| **Build/package** | npm is configured; package files are aligned; no Bun lockfile found. | Build was not verified: npm ci timed out and npm run build then failed because vite/client type definitions were missing from the incomplete install. This is not enough to establish a source-code compile failure. |

# 3. Critical findings — Version B introduces new risk

P0 means fix before deployment or before any real employee/payment data is used. These findings are directly visible in the newer source.

## P0 — Public backup export leaks records

File: `server.ts`, route `GET /api/backup/export`. No authentication or authorization middleware is applied. The response includes payment transactions, SMS outbox records and GCash transfer records; those records can include recipient mobile numbers and full payslip message contents.

**Required remark:** Restrict to a properly authenticated CEO/authorized backup role, require re-authentication for export, audit access, minimize data, and protect/download backups securely.

## P0 — Public backup restore can overwrite transactions

File: `server.ts`, route `POST /api/backup/restore`. No authentication, authorization, approval or robust schema validation is applied. A request can replace the in-memory transaction list with attacker-supplied values while the route returns a success message.

**Required remark:** Disable route until protected. Validate schema and ownership, require elevated authorization, take a verified pre-restore backup, and restore transactionally with integrity checks.

## P0 — PayMongo config routes are unauthenticated

Routes `GET/POST /api/paymongo/config` and `POST /api/paymongo/validate-config` have no authentication/authorization. Any caller reaching these routes can change the primary configuration/environment or add arbitrary customer identifiers and credentials.

**Required remark:** Require an authenticated privileged account, enforce customer tenancy and allowed IDs server-side, use secure secret storage, record audited changes, and add rate-limiting / CSRF protections as appropriate.

## P0 — Credential changes are not durable or customer-isolated in storage

The new config uses a process-memory Map and mutates process.env at runtime. Values are lost on process restart, and changing environment variables at runtime does not persist deployment configuration. The supplied customerId is user-controlled; no tenant authorization prevents one user changing another customer's config.

**Required remark:** Use a durable encrypted secrets/configuration store or deployment secret manager; derive tenant/customer access from authenticated identity, not a trusted request field.

## P0 — Automatic browser data purge can destroy records

File: `src/context/AppContext.tsx`. Version B switches the key to DERUEDA_ERP_V2, then on first launch removes V1 and V2 keys for employees, sites, attendance, payroll, payment records and related data. This can delete browser-held records instead of migrating them.

**Required remark:** Remove the automatic destructive cleanup. Export/back up first, create a deliberate migration that preserves unknown/real records, and use a separate explicit, confirmed action only for demo-data reset.

## P0 — GCash Cloudflare functions fabricate success (unchanged in both ZIPs)

Files: `functions/api/gcash/wallet.ts`, `disburse.ts`, and `batch-disburse.ts`. The wallet returns hardcoded "ACTIVE" balance figures; disbursement handlers generate random references and SVG receipts and return COMPLETED without contacting an official GCash payout API. The functions contain no visible authentication.

**Required remark:** Disable the endpoints or return clearly labelled simulation-only responses. Add server-side authorization and only mark paid after an official provider result or a carefully controlled, audited manual verification.

## P0 — PayMongo webhook is not actually verified (persists in both ZIPs)

The handler reads the PayMongo signature header but does not verify the signature against the raw request body. It accepts event payload fields, deduplicates in a process-memory Set, and treats any non-failed status as SUCCESS for a matching transaction.

**Required remark:** Verify official signatures with the correct webhook secret and timing-safe comparison; use durable unique event IDs, validate event type and amount/currency/reference, and reconcile against the provider API before finalizing.

### P0 — Production auth can be bypassed with role headers (persists in both ZIPs)

File: `server.ts`, `requireServerAuth`. When a valid session is absent, the middleware accepts role-related request headers and assigns a role based on their contents. Headers are client-controlled.

**Required remark:** Delete the fallback in production. Protected routes must require a valid server-issued session token and check role + site assignment from server-side session data only.

## 4. Critical payment/configuration remarks

**PayMongo configuration is currently a UI plus partial server-side scaffolding, not a production payment integration.**

| Issue | Evidence in Version B | Required correction |
|---|---|---|
| **Validation label is misleading** | The endpoint checks key prefixes (sk_test_ / sk_live_), public-key prefix, and a minimum webhook-secret length. It does not make an authenticated API request to PayMongo. Yet it says the credentials were "verified successfully." | Change message to "Format checks passed" or actually test the key with an official provider request and report the true result. |
| **Webhook secret variable mismatch** | .env.example declares PAYMONGO_WEBHOOK_SECRET_KEY, while the new server config reads PAYMONGO_WEBHOOK_SECRET. This means a correctly supplied .env.example variable may not load into the new config/webhook path. | Pick one canonical environment-variable name and use it everywhere, including deployment secrets, dialog, validator and webhook handler. |
| **Environment mismatch** | .env.example defaults PAYMONGO_ENVIRONMENT=sandbox; new dialog/server configuration expects test or live. Legacy handlers still default to sandbox. | Define one mapping: test/sandbox means non-live, live means production. Validate allowed values server-side and configure providers consistently. |
| **Masked public-key overwrite bug** | The config GET sends a partially masked public key that contains an ellipsis and trailing characters. The form treats that string as a real editable key; saving unchanged settings can store the masked placeholder as the public key because only bullet masks are skipped. | Never send masked placeholders back as credential values. Keep a separate "unchanged" state, and leave a secret unchanged unless a new value is explicitly entered. |
| **Persistence / client isolation** | The config Map is in-memory and defaults unknown customer IDs from the single primary process environment; there is no authoritative per-customer store. | Separate each customer's credentials and environment in a durable secrets store. Never fall back to the primary customer's secret for an unknown customer. |
| **PayMongo checkout/payouts still simulated** | Existing code creates local transaction IDs, uses a constructed checkout URL, simulates SUCCESS by timer, and marks records reconciled without an authoritative provider query. | Do not call these live PayMongo transactions. Create real provider checkout sessions/payments using their official APIs and verify signed webhooks + provider status before updating the ledger. |
| **New endpoints may not run on Cloudflare Pages** | The new config/backup handlers were added only to Express `server.ts`. The ZIP's Cloudflare `functions/` tree contains health, Gemini and GCash handlers but no matching PayMongo-config or backup handlers. | Confirm the production runtime. For Pages deployment, implement the required Cloudflare Pages Functions or deploy the Express service separately and point frontend API calls to that service. |

**Important: the dialog must not say "Server Protected" until all its endpoints actually enforce authentication. A UI label or hidden password field does not protect an API.**

## 5. Data, messaging, canteen, and public site

### Data handling and deletion

• Version B clears legacy localStorage data as a side effect of opening the application. This is not a safe migration strategy and could destroy a user's prior browser-held records.

• Most ERP state still lives in AppContext/localStorage; public inquiry and hiring handlers only append records to client state. They do not submit the data to a company server, so the company is not guaranteed to receive form submissions.

• Version B empties many seed arrays, which is good for a new deployment, but sample login-directory/equipment/audit records remain. Label retained examples as demo/test data and do not display them as real operations.

• Service worker caching rules should explicitly exclude private API and authenticated personal/financial responses; clear sensitive app caches on logout and do not treat localStorage as the authoritative payroll or payment ledger.

## Chatbox UI improvements / remarks

• The new screen better supports mobile conversation-list → chat navigation and has a details drawer. That is a useful UX improvement.

• Attachment progress is a timer simulation. The selected file is reduced to a name/size string when sendMessage is called; the actual file bytes are not uploaded or retained as an attachment. Do not present it as an uploaded file until a protected upload API and durable storage exist.

• Messages still use AppContext state/localStorage and have no demonstrated persistent shared backend, so multiple devices/users will not have reliable synchronization.

• The screen labels the channel “Verified Enterprise Channel”; remove or soften that claim until encryption, authorization and any promised verification have actually been implemented and tested.

## Canteen GCash and site settings

• The per-site canteen destination field remains in the data model and the interface displays a default number if none is configured. Replace the fallback with “Not configured” to avoid accidentally routing payment to a generic number.

• The source still contains legacy wording/options for cash and GCash; enforce GCash-only on the server-side checkout and relevant records, not only in the UI.

• Site settings still rely on client-side state/localStorage. A per-site destination must be stored server-side with authorization: site staff can change only assigned site settings; CEO can oversee all sites; every change is audited.

• A screenshot or click must never prove payment. Only verified provider confirmation or a documented authorized manual verification should change the ledger status.

## Public files and release configuration

• Added robots.txt, sitemap.xml and security.txt are positive additions, but they reference deruedaconstruction.com. Confirm that domain, policy URL, contact email and security contact are controlled and live before publishing those claims.

• manifest.json references /favicon.ico, but the ZIP has no favicon file. index.html still lacks a favicon declaration and social preview image metadata.

• Security headers added to Express server.ts do not automatically apply to a static Cloudflare Pages deployment. public/_headers still has only nosniff and referrer policy; set the actual deployment headers at the correct hosting layer.

• The public/_redirects file uses a broad SPA fallback. Verify API and static paths return the correct JSON/content types rather than index.html in the deployed environment.

# 6. Build and testing status

| Check | Observed result | Remark |
|---|---|---|
| npm package management | package.json and package-lock.json root name/version align in both archives; no Bun lockfile found. | Pass at metadata level. |
| npm install / dependency restore | npm ci did not complete; the command timed out in this environment. | Blocked by dependency-install/network timeout; rerun in a network-enabled build environment. |
| npm run build | Exited with TS2688: Cannot find type definition file for vite/client after incomplete node_modules installation. | Build gate has NOT passed. This is inconclusive for source correctness because dependencies were unavailable. |
| Unit tests / lint | package.json scripts test and lint both run tsc --noEmit; no separate test runner configured. | Add real tests for auth, backups, checkout, webhook validation, site isolation and data migration. |
| Live site recheck | Browser reached ERR_QUIC_PROTOCOL_ERROR on emybuilder.pages.dev and unrelated control sites. | Inconclusive, not evidence the site itself is offline. Verify again once network path works. |

# 7. New and changed files

Compared project trees: Version A contains 111 files under the project root; Version B contains 118. Version B adds 7 files, changes 22 existing files, and removes none. package.json and package-lock.json were not changed.

## New files

- `src/components/payments/PayMongoEnvConfigDialog.tsx`
- `public/robots.txt`
- `public/sitemap.xml`
- `public/manifest.json`
- `public/.well-known/security.txt`
- `public/assets/aistudio/finishing_upgrades_48893452.pdf`
- `.aistudio/.../.tempmediaStorage/media_1792666816059_01aee09a.pdf`

## Important modified files

`server.ts, src/context/AppContext.tsx, src/utils/credentials.ts, src/server/auth.ts (unchanged; reviewed for inherited risk), src/data/initialData.ts, src/components/payments/PayMongoEnvConfigDialog.tsx (new), src/components/views/MessagesView.tsx, src/components/views/SettingsView.tsx, src/components/views/PaymentRecordsHubView.tsx, src/components/views/CanteenView.tsx, src/components/layout/Sidebar.tsx, index.html, plus 10 other changed UI/component files`

# 8. Recommended fix order / final remarks

## P0 — Remove exposure and protect access

Immediately secure or disable /api/backup/export, /api/backup/restore, and /api/paymongo/config endpoints. Remove the role-header fallback. Keep real GCash disbursement routes disabled until official integration and authorization exist.

## P0 — Stop data deletion

Replace the automatic V1/V2 localStorage wipe with a backup-first migration. Never delete user data implicitly while opening the app.

## P0 — Stop fake financial success

Disable fabricated wallet balances, receipt generation that claims money was sent, timed PayMongo success, fake checkout URLs, and fake reconciliation. Clearly mark any mock mode.

## P1 — Implement customer-specific PayMongo correctly

Use authenticated tenant-admin configuration, durable secret storage, consistent variable names, real provider API validation, official checkout-session creation, signature-verified webhooks, durable idempotency and provider reconciliation.

## P1 — Persist core ERP records

Provide a shared authoritative backend/database for employees, attendance, payroll, site GCash destinations, messages, forms, transactions and audit history. Protect personal data and private API caches.

## P2 — Finish UI & release quality

Keep the improved chatbox layout but implement real file upload/storage; enforce GCash-only server-side; remove generic canteen fallback numbers; verify static metadata, correct domain links, mobile layout and accessibility.

## P2 — Pass build and runtime verification

Run npm ci successfully, npm run build, automated tests, security checks, and desktop/mobile smoke tests. Verify the actual deployed build and all relevant API routes before declaring completion.

**Final verdict: Version B is a functional/UI iteration, not a safe production release. Keep the improved front-end credential handling, empty clean-install data, simplified navigation and chatbox layout. Do not ship its new config/backup routes or the destructive storage cleanup as written. First fix the P0 findings, then repeat the source and live deployment audits. No source archive or deployed application was modified during this comparison.**